Detecting Illicit Bitcoin Transactions with Label-Consistent Graph Stacking

BT4012 In-class Kaggle Competition 2026 · e1304439 · Code: github.com/emmyyyli/BT4012-Kaggle-Competition (submission notebook `e1304439.ipynb`)

1. Introduction

Criminals use Bitcoin to launder money and pay for illegal goods. The competition asks for a model that ranks Bitcoin transactions by their probability of being illicit. The data is the Elliptic transaction graph: each node is a transaction described by 165 anonymised features, and each edge is a payment flow, where an output of one transaction is spent by another. Models train on time steps 1–35 and are scored by ROC-AUC on the later steps 36–49. This mirrors anti-money-laundering (AML) work: investigators need a risk ranking that catches illicit flows early without flooding them with false alarms.

The problem has three properties, and each one shaped the approach:

- **Scarce, imbalanced labels.** Only 31,235 of 141,772 training transactions are labeled, and 11.7% of those are illicit. Supervised learning therefore uses labeled rows only, with balanced class weights.
- **Temporal drift.** The test period lies in the future; a classifier separates training rows from test rows with AUC ≈ 1.0 from the features alone. Validation is therefore always forward in time, training is weighted toward recent steps, and `time_step` is never a model input.
- **Graph structure with missing context.** Neighbouring transactions are informative, but in the test period most neighbours have no features. Every graph feature is therefore built from information that exists in the same form at training and prediction time.

The final model is a two-stage gradient-boosted stack with label-free, *label-consistent* neighbourhood features. It raised the public ROC-AUC from 0.94246 (first submission) to **0.96814**, within noise of the leaderboard top (0.97012).

2. Methodology

2.1 Data preprocessing and feature engineering

Audit. The supplied features have no missing values and no duplicate IDs. `feat_1`–`feat_93` describe the transaction itself ("local"); `feat_94`–`feat_165` aggregate its one-hop neighbourhood, as documented for Elliptic. Rows labeled `none` (110,537) were treated as **unlabeled, not licit**. Using them as pseudo-labels lowered time-forward AUC by about 0.03, because unlabeled transactions are not a random sample of labeled ones. `time_step` is used only to split data. As a feature it lowered long-horizon AUC (0.878 vs 0.884), and trees cannot extrapolate a counter to unseen steps.

The decisive finding: the test period has blank neighbourhoods.

- The 15,329 test rows are exactly the *labeled* transactions of steps 36–49: 31,235 + 15,329 equals the documented Elliptic labeled total of 46,564.
- The 46,668 edge-list nodes without features are the *unlabeled* transactions of those steps.
- So **65.6% of test-node edges lead to featureless nodes, against 0% in training.** No edge crosses time steps.

A graph neural network that reads all neighbours therefore sees a different input distribution at test time. Our GraphSAGE blend dropped the public score from 0.94246 to 0.91059.

Feature drift. Medians of 19.4% of the aggregated features shift by more than one interquartile range from training to test, against 9.7% within the training period. Local features barely move (1.1%). An aggregated-only model scored 0.880 on Kaggle. Removing the five most-drifted features was neutral (0.94365 vs 0.94417), so all 165 features were kept. Correlated-cluster ablation, SHAP-stability subsets and adversarial feature dropping gave no gain either, so there is no explicit feature selection.

Label-consistent graph features.

- **Motivation:** labeled transactions are strongly homophilous. A labeled neighbour of an illicit transaction is illicit 57.4% of the time, against a 11.7% base rate.
- **Design rule:** context is built only from *labeled* neighbours, the only transactions with features in both periods, and no target label is ever used. With this rule, coverage is similar in training and test (Figure 1). Features, each added as a separately tested step:

1. **Neighbour scores:** mean, max, incoming-mean and outgoing-mean of stage-1 predictions over labeled neighbours.
2. **Neighbourhood structure:** in/out degree, plus the number and share of labeled neighbours.
3. **Neighbour feature statistics:** mean, max, std, incoming-mean and outgoing-mean over labeled neighbours of the 25 most influential supplied features, ranked by test-row SHAP of the base model (no labels used).
4. **Two-hop context:** walk-weighted means of stage-1 scores and five key features over labeled nodes two steps away. Walks may pass *through* featureless nodes, because the edge list is complete in both periods.

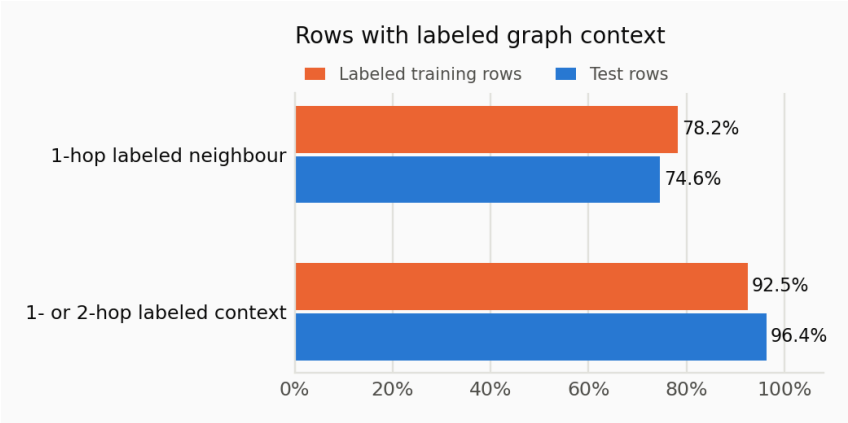


Figure 1. Share of rows with labeled graph context. The one-hop rule keeps training and test comparable; two-hop walks through featureless nodes reach 21.8% of test rows that have no labeled neighbour (14.3% in training).

2.2 Model development and validation

Base learner. LightGBM, tuned with Optuna (40 trials): 9 leaves, 500 trees, learning rate 0.025, at least 49 rows per leaf, 64% of features and 98% of rows per tree, L2 penalty 2.0. Every search, including XGBoost and CatBoost, preferred shallow trees (4–9 leaves or depth 3–4): regularisation is the main defence against drift. Class imbalance is handled with `class_weight="balanced"`; ROC-AUC needs no threshold, so no resampling.

Stacking without leakage.

1. **Stage 1:** score every labeled training row *out-of-fold*, with folds as blocks of consecutive time steps. Score test rows with the mean of the five fold models, so that training and test scores come from the same kind of model.
2. **Context:** build the graph features of §2.1 from those scores.
3. **Stage 2:** refit LightGBM on the supplied features plus the context.
4. **Recency and seeds:** both stages weight rows by 2^((t – 35)/14), so step 35 counts 1 and step 21 counts 0.5. Predictions are averaged over 10 seeds.

Validation. The original 4-fold time-series CV averaged 0.977, while Kaggle gave 0.942. Single-holdout scores also moved by up to 0.02 between random seeds, which was larger than most of the effects being tested. We therefore built a harness (`src/evaluation.py`):

- five forward windows (train ≤ t, validate a later block), each with five seeds;
- seed-averaged ROC-AUC and PR-AUC, per-step AUC, and a paired bootstrap against a reference;
- adoption rule: the mean must improve and no window may lose more than 0.01.

It had one limitation. The long windows (≈0.90 AUC) match the test's difficulty but ranked models in the *opposite* order to Kaggle. The short windows (≈0.996) ranked them correctly but are too close to the ceiling to resolve gains of 0.01. Local validation therefore served as a correctness check, and the public leaderboard (30% of the test set) acted as the final arbiter. To limit overfitting to it, every submission tested one stated hypothesis against a fixed reference, and a switch required ≥ +0.003 (about two paired standard errors).

2.3 Model exploration and final strategy

Table 1. Models explored. "Local" is the mean ROC-AUC over the five validation windows, relative to the reference used at the time.

Approach	Local evidence	Public AUC	Decision
LightGBM + `time_step` + `in_degree` (first submission)	—	0.94246	baseline
Random Forest	below LightGBM on every window	—	rejected
Tuned shallow LightGBM, 165 features	+0.010 vs deep LightGBM	0.94417	base learner
XGBoost / CatBoost / cross-family ensembles	no gain over LightGBM	—	rejected
Aggregated features only	best on long windows	0.88023	rejected (drift)
GraphSAGE blend (all neighbours)	—	0.91059	rejected (blank neighbours)
GraphSAGE with test-like masking	−0.047	—	rejected
Self-supervised graph embeddings	−0.04 (long holdout)	—	rejected
Pseudo-labeling `none` rows	−0.03	—	rejected
Recency weights (half-life 14)	≈+0.001 short windows	0.95066	adopted
Labeled-neighbour stacking	failed local rule	0.95389	adopted
+ structure & neighbour features / + recency		0.95871 / 0.96342	adopted

Approach	Local evidence	Public AUC	Decision
+ two-hop context		0.96668	adopted
+ 25-feature neighbour statistics (**final**)		**0.96814**	final model
3-hop score propagation	best long-window variant	0.96108	rejected
Extra round of collective classification	+0.005–0.011 long windows	0.96698	rejected (within noise)
Blend with a non-graph model		0.95328	rejected
Final model with 30 seeds / half-life 21	test rank ρ ≥ 0.998	0.96785 / 0.96670	stability check
Rank blend of four graph models		0.96686	private pick 2

Two patterns decided the final strategy:

- **Added complexity without new information never helped.** This covers the extra model families, blends with non-graph models, deeper propagation and an extra round of collective classification.
- **New information, computed the same way in training and test, always did.** This covers neighbours, their features and two-hop reach. Recency weighting helped independently, twice.

Final picks. Kaggle scores the better of two selected submissions on the private 70%:

- **Pick 1:** the final model (0.96814), which `e1304439.ipynb` reproduces exactly.
- **Pick 2:** a rank blend of the four best graph models (0.96686), which averages over feature configurations as insurance against the private split.

3. Results and discussion

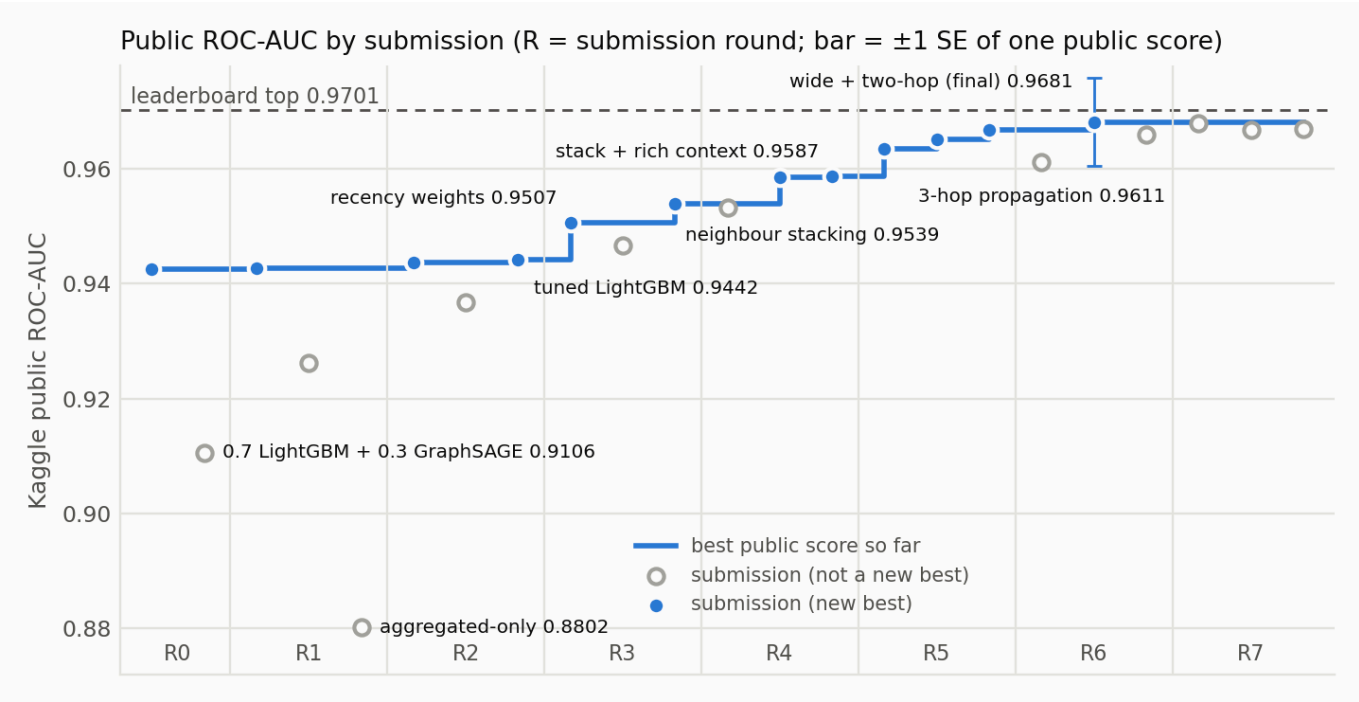


Figure 2. Public ROC-AUC of every submission. The best-so-far line rises only when new, label-consistent information is added; the ±1 SE bar (0.0077, about 270 illicit public rows) shows that the final gap to the leader is within noise.

Table 2. Final model vs the non-graph base learner on the two validation windows whose ranking matched Kaggle (3 seeds). P = precision. Validation windows are about 18% illicit, so operating points adapt to prevalence.

Window (illicit / rows)	Model	ROC-AUC	PR-AUC	P = recall at k = #illicit	P at 95% recall	Best F1
train ≤25 → 26–35 (1,307 / 7,035)	non-graph	0.9954	0.9820	0.941	0.917	0.942
	final	**0.9971**	**0.9895**	**0.953**	**0.962**	**0.957**
train ≤28 → 29–35 (1,102 / 6,028)	non-graph	0.9960	0.9809	0.946	0.940	0.947
	final	**0.9982**	**0.9933**	**0.966**	**0.979**	**0.966**

What the numbers mean.

- **On Kaggle,** graph context added +0.024 public AUC over the base learner (0.94417 → 0.96814). The largest single steps were neighbour stacking (+0.0097) and two-hop context (+0.0033 over its parent).
- **In triage terms** (Table 2), when 95% of illicit transactions are caught, false alarms among flagged transactions fall from 8.3% to 3.8% and from 6.0% to 2.1%. That roughly halves wasted investigator effort.
- **The model is stable.** Tripling the seeds or changing the recency half-life by 50% barely changes the test ranking (Spearman ρ ≥ 0.998).

- **Why two-hop context helped so much more on test than locally:** it reaches 21.8% of test rows that have no labeled neighbour, against 14.3% in training (Figure 1).

Limitations.

- No validation window inside steps 1–35 reproduces the train→test drift or the test period's sparse labels. Graph variants *lost* on long local windows yet won on Kaggle, so close calls relied on the public leaderboard.
- The public leaderboard covers about 4,600 rows, so one public AUC has a standard error of about 0.008. With about 23 submissions compared on it, the best public score is slightly optimistic. Each step tested one stated idea against a fixed reference to keep this selection bias small.
- Anonymised features prevent a domain explanation (amounts and fees are unknown).
- The documented dark-market shutdown at step 43 (Weber et al., 2019) falls in the hidden test period and cannot be analysed directly.

4. Conclusion and reflection

The main lesson is that **how information is made available matters more than model complexity.**

- **What plateaued or failed:** a shallow LightGBM on the supplied features plateaued at 0.944. Neural graph models and embeddings failed because the neighbourhoods they read in training do not exist at test time.
- **What worked:** restricting graph features to labeled neighbours, the information that will genuinely exist at prediction time, turned graph structure from a liability into the largest source of gain. Recency weighting, out-of-fold stacking and disciplined, single-hypothesis validation made the gains reliable.
- **What I would improve:** design validation windows that mask labels inside the validation period to imitate the test regime, and test directional (forward vs backward fund-flow) two-hop features.
- **Wider lesson:** for AI-driven fraud detection, a feature is only as good as its availability in production. The same "same information at training and serving time" discipline that prevented leakage here is what keeps a deployed AML model honest.

References. Weber et al. (2019), *Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics*, KDD workshop. Ke et al. (2017), *LightGBM*, NeurIPS. Akiba et al. (2019), *Optuna*, KDD. Lu & Getoor (2003), *Link-based Classification*, ICML. Lorenz et al. (2020), *ML methods to detect money laundering in the Bitcoin blockchain in the presence of label scarcity*, ICAIF. Hanley & McNeil (1982), *The meaning and use of the area under a ROC curve*, Radiology.